```
"""
Discord 荒らし対策Bot (Python / discord.py版)
機能: /antitroll on|off|status, 招待リンク許可チャンネル管理,
      連投/メンションスパム検知(自動タイムアウト+解除/BANボタン), レイド検知
"""
import os
import re
from datetime import datetime, timedelta, timezone

import discord
from discord import app_commands
from discord.ext import commands, tasks
from dotenv import load_dotenv

import config
import settings_store as settings

load_dotenv()

TOKEN = os.getenv("DISCORD_TOKEN")
GUILD_ID = os.getenv("GUILD_ID")  # 任意: 指定すると起動時にそのサーバーへ即時コマンド反映
LOG_CHANNEL_ID = os.getenv("LOG_CHANNEL_ID")
QUARANTINE_ROLE_ID = os.getenv("QUARANTINE_ROLE_ID")

INVITE_REGEX = re.compile(
    r"(discord\.gg|discord(?:app)?\.com/invite)/[a-zA-Z0-9-]+", re.IGNORECASE
)

intents = discord.Intents.default()
intents.message_content = True
intents.members = True
intents.guilds = True

bot = commands.Bot(command_prefix="!", intents=intents)

# user_id -> {"timestamps": [float,...], "last_content": str, "dup_count": int}
message_history: dict[int, dict] = {}
# guild_id -> [float,...]
join_history: dict[int, list] = {}
# guild_id -> expire_timestamp(float)
raid_lockdown: dict[int, float] = {}


# ==============================
```

```
# ユーティリティ
# ==============================
def is_exempt(member: discord.Member) -> bool:
    if member is None:
        return False
    if config.WHITELIST["exempt_admins"] and member.guild_permissions.administrat
        return True
    exempt_role_ids = config.WHITELIST["exempt_role_ids"]
    if exempt_role_ids:
        return any(r.id in exempt_role_ids for r in member.roles)
    return False


def get_log_channel(guild: discord.Guild):
    if not LOG_CHANNEL_ID:
        return None
    return guild.get_channel(int(LOG_CHANNEL_ID))


async def send_log(guild: discord.Guild, embed: discord.Embed):
    channel = get_log_channel(guild)
    if channel is None:
        return
    try:
        await channel.send(embed=embed)
    except discord.HTTPException:
        pass


async def safe_delete(message: discord.Message):
    try:
        await message.delete()
    except discord.HTTPException:
        pass


async def timeout_member(member: discord.Member, ms: int, reason: str) -> bool:
    try:
        await member.timeout(timedelta(milliseconds=ms), reason=reason)
        return True
    except (discord.Forbidden, discord.HTTPException) as err:
        print(f"タイムアウト処理エラー: {err}")
        return False


def build_moderation_view(user_id: int) -> discord.ui.View:
    view = discord.ui.View(timeout=None)
```

```
    view.add_item(
        discord.ui.Button(
            label="タイムアウト解除",
            style=discord.ButtonStyle.success,
            custom_id=f"mod_untimeout_{user_id}",
        )
    )
    view.add_item(
        discord.ui.Button(
            label="BAN",
            style=discord.ButtonStyle.danger,
            custom_id=f"mod_ban_{user_id}",
        )
    )
    return view


async def send_moderation_prompt(
    message: discord.Message, member: discord.Member, title: str, reason_text: st
):
    embed = discord.Embed(
        title=f"🚨 {title}",
        description=reason_text,
        color=discord.Color.red(),
        timestamp=datetime.now(timezone.utc),
    )
    embed.add_field(name="ユーザー", value=f"{member.mention} ({member.id})", inli
    embed.add_field(name="チャンネル", value=message.channel.mention, inline=True)

    view = build_moderation_view(member.id)
    log_channel = get_log_channel(message.guild)
    target = log_channel or message.channel
    try:
        await target.send(embed=embed, view=view)
    except discord.HTTPException:
        pass


async def warn_user(message: discord.Message, reason: str):
    embed = discord.Embed(
        title="⚠️ メッセージを削除しました",
        description=reason,
        color=discord.Color.orange(),
        timestamp=datetime.now(timezone.utc),
    )
    embed.add_field(name="ユーザー", value=message.author.mention, inline=True)
    await send_log(message.guild, embed)
```

```
# ==============================
# イベント：起動時
# ==============================
@bot.event
async def on_ready():
    print(f"ログイン完了：{bot.user}")
    try:
        if GUILD_ID:
            guild_obj = discord.Object(id=int(GUILD_ID))
            bot.tree.copy_global_to(guild=guild_obj)
            synced = await bot.tree.sync(guild=guild_obj)
            print(f"ギルド({GUILD_ID})にコマンドを{len(synced)}件同期しました(即時反映)")
        else:
            synced = await bot.tree.sync()
            print(f"グローバルコマンドを{len(synced)}件同期しました(反映まで最大1時間)")
    except discord.HTTPException as err:
        print("コマンド同期エラー:", err)

    if not cleanup_task.is_running():
        cleanup_task.start()


# ==============================
# イベント：メッセージ監視(招待リンク/メンション/スパム)
# ==============================
@bot.event
async def on_message(message: discord.Message):
    try:
        if message.author.bot or message.guild is None:
            return
        if not settings.is_enabled(message.guild.id):
            return

        member = message.author
        if is_exempt(member):
            return

        now = datetime.now(timezone.utc).timestamp()
        user_id = member.id

        # --- 招待リンクフィルター(許可チャンネル以外は削除) ---
        if config.INVITE_FILTER["enabled"] and INVITE_REGEX.search(message.conten
            allowed = settings.is_invite_allowed_in_channel(
                message.guild.id, message.channel.id
            )
```

```
    if not allowed:
        await safe_delete(message)
        await warn_user(
            message,
            "このチャンネルでは招待リンクの投稿は許可されていません。"
            "(許可チャンネルは `/antitroll invite allow` で設定できます)",
        )
        return

# --- メンション荒らし対策 ---
if config.MENTION_SPAM["enabled"]:
    mention_count = len(message.mentions) + len(message.role_mentions)
    if mention_count >= config.MENTION_SPAM["max_mentions"]:
        if config.SPAM["delete_messages"]:
            await safe_delete(message)
        timed_out = await timeout_member(
            member, config.MENTION_SPAM["timeout_ms"], "メンション荒らし検知"
        )
        if timed_out:
            minutes = config.MENTION_SPAM["timeout_ms"] // 60000
            await send_moderation_prompt(
                message,
                member,
                "メンション荒らしを検知",
                f"{member.mention} を{minutes}分間タイムアウトしました。"
                f"(メンション数: {mention_count})\n"
                "必要に応じて下のボタンで解除/BANしてください。",
            )
        return

# --- 連投・スパム検知 ---
if config.SPAM["enabled"]:
    history = message_history.setdefault(
        user_id, {"timestamps": [], "last_content": "", "dup_count": 0}
    )
    interval_sec = config.SPAM["interval_ms"] / 1000
    history["timestamps"] = [
        t for t in history["timestamps"] if now - t < interval_sec
    ]
    history["timestamps"].append(now)

    if message.content and message.content == history["last_content"]:
        history["dup_count"] += 1
    else:
        history["dup_count"] = 1
        history["last_content"] = message.content
```

```
            is_flood = len(history["timestamps"]) > config.SPAM["max_messages"]
            is_duplicate = history["dup_count"] >= config.SPAM["duplicate_thresho

            if is_flood or is_duplicate:
                if config.SPAM["delete_messages"]:
                    await safe_delete(message)
                reason = "連投スパム検知" if is_flood else "同一メッセージ連投検知"
                timed_out = await timeout_member(
                    member, config.SPAM["timeout_ms"], reason
                )
                if timed_out:
                    minutes = config.SPAM["timeout_ms"] // 60000
                    detail = "短時間の連投" if is_flood else "同一メッセージの連投"
                    await send_moderation_prompt(
                        message,
                        member,
                        "スパム行為を検知",
                        f"{member.mention} を{minutes}分間タイムアウトしました。"
                        f"(理由: {detail})\n"
                        "必要に応じて下のボタンで解除/BANしてください。",
                    )
                history["timestamps"] = []
                history["dup_count"] = 0

    except Exception as err:  # noqa: BLE001
        print("on_message処理エラー:", err)


# ==============================
# イベント: 大量参加(レイド)検知
# ==============================
@bot.event
async def on_member_join(member: discord.Member):
    try:
        if not config.ANTI_RAID["enabled"]:
            return
        if not settings.is_enabled(member.guild.id):
            return

        guild_id = member.guild.id
        now = datetime.now(timezone.utc).timestamp()
        interval_sec = config.ANTI_RAID["join_interval_ms"] / 1000

        history = [
            t for t in join_history.get(guild_id, []) if now - t < interval_sec
        ]
        history.append(now)
```

```
        join_history[guild_id] = history

        lockdown_expire = raid_lockdown.get(guild_id)
        in_lockdown = lockdown_expire is not None and lockdown_expire > now

        if len(history) >= config.ANTI_RAID["join_threshold"] or in_lockdown:
            if not in_lockdown:
                lockdown_sec = config.ANTI_RAID["lockdown_ms"] / 1000
                raid_lockdown[guild_id] = now + lockdown_sec
                embed = discord.Embed(
                    title="🚨 大量参加(レイド)を検知",
                    description=(
                        f"{int(interval_sec)}秒間に{len(history)}人が参加しました。"
                        f"{int(lockdown_sec // 60)}分間、新規参加者を自動処理します。"
                    ),
                    color=discord.Color.red(),
                    timestamp=datetime.now(timezone.utc),
                )
                await send_log(member.guild, embed)

            if config.ANTI_RAID["action"] == "kick":
                try:
                    await member.kick(reason="レイド対策：自動キック")
                except discord.HTTPException:
                    pass
            else:
                if QUARANTINE_ROLE_ID:
                    role = member.guild.get_role(int(QUARANTINE_ROLE_ID))
                    if role:
                        try:
                            await member.add_roles(role, reason="レイド対策：隔離ロ-
                        except discord.HTTPException:
                            pass
    except Exception as err:  # noqa: BLE001
        print("on_member_join処理エラー:", err)


# ==============================
# イベント：モデレーションボタン操作
# ==============================
@bot.event
async def on_interaction(interaction: discord.Interaction):
    if interaction.type != discord.InteractionType.component:
        return
    custom_id = interaction.data.get("custom_id", "") if interaction.data else ""
    if not custom_id.startswith("mod_"):
        return
```

```
perms = (
    interaction.channel.permissions_for(interaction.user)
    if interaction.channel
    else None
)
has_permission = bool(
    perms and (perms.moderate_members or perms.administrator)
)
if not has_permission:
    await interaction.response.send_message(
        "このボタンを操作する権限がありません。", ephemeral=True
    )
    return

try:
    _, action, user_id_str = custom_id.split("_", 2)
    user_id = int(user_id_str)
except ValueError:
    return

guild = interaction.guild
try:
    target_member = guild.get_member(user_id) or await guild.fetch_member(use
except discord.NotFound:
    await interaction.response.send_message(
        "対象ユーザーがサーバーに見つかりませんでした。", ephemeral=True
    )
    return

try:
    if action == "untimeout":
        await target_member.timeout(None, reason=f"解除実行者: {interaction.use
        await interaction.response.send_message(
            f"✅ {target_member.mention} のタイムアウトを {interaction.user.ment
        )
    elif action == "ban":
        await target_member.ban(
            reason=f"BAN実行者: {interaction.user}(荒らし対策ボタン経由)"
        )
        await interaction.response.send_message(
            f"🔨 {target_member} を {interaction.user.mention} がBANしました。"
        )
    else:
        return

    # 操作後はボタンを無効化してこれ以上押せないようにする
```

```
        disabled_view = discord.ui.View(timeout=None)
        disabled_view.add_item(
            discord.ui.Button(
                label="タイムアウト解除",
                style=discord.ButtonStyle.success,
                custom_id=f"mod_untimeout_{user_id}",
                disabled=True,
            )
        )
        disabled_view.add_item(
            discord.ui.Button(
                label="BAN",
                style=discord.ButtonStyle.danger,
                custom_id=f"mod_ban_{user_id}",
                disabled=True,
            )
        )
        if interaction.message:
            await interaction.message.edit(view=disabled_view)

    except discord.Forbidden:
        await interaction.response.send_message(
            "Botの権限不足のため操作できませんでした。(ロール順位を確認してください)",
            ephemeral=True,
        )
    except discord.HTTPException as err:
        await interaction.response.send_message(
            f"操作に失敗しました: {err}", ephemeral=True
        )


# ==============================
# スラッシュコマンド: /antitroll
# ==============================
antitroll_group = app_commands.Group(
    name="antitroll",
    description="荒らし対策Botの設定",
    default_permissions=discord.Permissions(manage_guild=True),
)
invite_group = app_commands.Group(
    name="invite",
    description="招待リンクの許可チャンネル設定",
    parent=antitroll_group,
)


@antitroll_group.command(name="on", description="このサーバーで荒らし対策を有効化します'
```

```
async def antitroll_on(interaction: discord.Interaction):
    settings.set_enabled(interaction.guild.id, True)
    await interaction.response.send_message("✅ 荒らし対策を**有効化**しました。")


@antitroll_group.command(name="off", description="このサーバーで荒らし対策を無効化します
async def antitroll_off(interaction: discord.Interaction):
    settings.set_enabled(interaction.guild.id, False)
    await interaction.response.send_message("🛑 荒らし対策を**無効化**しました。")


@antitroll_group.command(name="status", description="現在の設定状況を表示します")
async def antitroll_status(interaction: discord.Interaction):
    s = settings.get_guild_settings(interaction.guild.id)
    channel_list = ", ".join(f"<#{cid}>" for cid in s["allowed_invite_channels"])
    embed = discord.Embed(
        title="荒らし対策Bot ステータス",
        color=discord.Color.green() if s["enabled"] else discord.Color.greyple(),
    )
    embed.add_field(name="有効状態", value="✅ 有効" if s["enabled"] else "🛑 無効"
    embed.add_field(name="招待リンク許可チャンネル", value=channel_list, inline=False
    await interaction.response.send_message(embed=embed)


@invite_group.command(name="allow", description="指定チャンネルで招待リンクの投稿を許可
@app_commands.describe(channel="許可するチャンネル")
async def invite_allow(interaction: discord.Interaction, channel: discord.TextCha
    settings.allow_invite_channel(interaction.guild.id, channel.id)
    await interaction.response.send_message(f"✅ {channel.mention} で招待リンクの投稿


@invite_group.command(name="disallow", description="指定チャンネルの招待リンク許可を解除
@app_commands.describe(channel="解除するチャンネル")
async def invite_disallow(interaction: discord.Interaction, channel: discord.Text
    settings.disallow_invite_channel(interaction.guild.id, channel.id)
    await interaction.response.send_message(f"🛑 {channel.mention} の招待リンク許可


@invite_group.command(name="list", description="招待リンクが許可されているチャンネル一覧
async def invite_list(interaction: discord.Interaction):
    s = settings.get_guild_settings(interaction.guild.id)
    channel_list = ", ".join(f"<#{cid}>" for cid in s["allowed_invite_channels"])
    await interaction.response.send_message(f"招待リンク許可チャンネル: {channel_list


bot.tree.add_command(antitroll_group)
```

```
# ==============================
# 定期クリーンアップ(メモリリーク防止)
# ==============================
@tasks.loop(seconds=30)
async def cleanup_task():
    now = datetime.now(timezone.utc).timestamp()
    interval_sec = config.SPAM["interval_ms"] / 1000
    for user_id in list(message_history.keys()):
        history = message_history[user_id]
        history["timestamps"] = [t for t in history["timestamps"] if now - t < in
        if not history["timestamps"]:
            del message_history[user_id]
    for guild_id in list(raid_lockdown.keys()):
        if raid_lockdown[guild_id] < now:
            del raid_lockdown[guild_id]


if __name__ == "__main__":
    if not TOKEN:
        raise SystemExit("DISCORD_TOKEN が設定されていません。.env を確認してください。"

    # RenderのWeb Service(無料プラン)はPORT環境変数を自動で渡してくる。
    # 検知した場合のみ、スリープ対策用の簡易HTTPサーバーを起動する。
    # Background Worker(有料)やローカル実行時はPORTが無いため何もしない。
    if os.getenv("PORT"):
        from keep_alive import keep_alive

        keep_alive()

    bot.run(TOKEN)
```